

INTERFACE

- 1- C1 EXTENDS C2
- 2-C1 IMPLEMENTS I1
- 3- C1 IMPLEMENTS I1 I2
- 4- C1 EXTENDS C2 IMPLEMENTS I1
- 5- I1 EXTENDS I2
- 6-I1 EXTENDS I2 I3

1- C1 EXTENDS C2

```
class C2 {  
    void showC2() {  
        System.out.println("This is C2");  
    }  
}  
  
class C1 extends C2 {  
    void showC1() {  
        System.out.println("This is C1");  
    }  
}  
  
public class C1ExtC2 {  
    public static void main(String[] args) {  
        C1 obj = new C1();  
  
        obj.showC1();  
        obj.showC2();  
    }  
}
```

Explanation

- `class C2` is the **parent (super) class**.
- `showC2()` is a method of class C2.
- `class C1 extends C2` means **C1 inherits C2**.
- Therefore, C1 can access the methods of C2.
- `showC1()` is a method of C1.

- `C1 obj = new C1();` creates an object of C1.
- `obj.showC1();` calls the method of C1.
- `obj.showC2();` calls the inherited method of C2.

Inheritance Structure

C2

↑

C1

Output

This is C1

This is C2

```
Storage\c4b8e345e3d9487287b73b8a2b53a4dd\redhat.java\jdk_ws\Assignment_2_c88ce236\bin - C1ExtC2
This is C1
This is C2
```

2-C1 IMPLEMENTS I1

```
interface I1 {
    int x = 100;

    void display();
    void show();
}

class C1 implements I1 {
    @Override
    public void display() {
        System.out.println("It is from interface");
    }

    @Override
    public void show() {
        System.out.println("Method from interface");
    }
}

public class C1Impl1 {
    public static void main(String[] args) {
        C1 obj = new C1();
    }
}
```

```

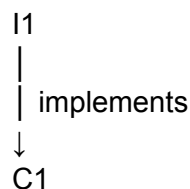
        obj.display();
        obj.show();
        System.out.println(obj.x);
    }
}

```

Explanation

- `interface I1` creates an interface named **I1**.
- `int x = 100` is a variable in the interface. Interface variables are automatically **public, static, and final**.
- `void display()` and `void show()` are abstract methods declared in I1.
- `class C1 implements I1` means **C1 implements the I1 interface**.
- C1 must provide implementations for both `display()` and `show()`.
- `public` is required because interface methods are public by default.
- `@Override` indicates that the method is implementing a method declared in the interface.
- `C1 obj = new C1();` creates an object of C1.
- `obj.display()` calls the implemented `display()` method.
- `obj.show()` calls the implemented `show()` method.
- `obj.x` accesses the interface variable whose value is `100`.

Structure



Output

It is from interface
Method from interface
100

```

PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> javac C1ImpI1.java
PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> java C1ImpI1
it is from interface
method from interface
100

```

3-C1 IMPLEMENTS I1 I2 I3

```
interface I1 {  
    int x = 20;  
  
    void display();  
}
```

```
interface I2 {  
    int y = 50;  
  
    void play();  
}
```

```
interface I3 {  
    int z = 30;  
  
    void show();  
}
```

```
class C1 implements I1, I2, I3 {  
  
    public void display() {  
        System.out.println(x);  
    }  
  
    public void play() {  
        System.out.println("It is from interface 2");  
    }  
  
    public void show() {  
        System.out.println(z);  
    }  
}
```

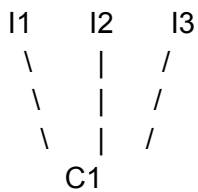
```
public class C1Impl1I2I3 {  
    public static void main(String[] args) {  
  
        C1 obj = new C1();  
  
        obj.display();  
        obj.play();  
        obj.show();  
        System.out.println(obj.y);  
    }  
}
```

```
}  
}
```

Explanation

- **I1**, **I2**, and **I3** are three different interfaces.
- **I1** contains variable **x = 20** and method **display()**.
- **I2** contains variable **y = 50** and method **play()**.
- **I3** contains variable **z = 30** and method **show()**.
- **class C1 implements I1, I2, I3** means **C1 implements all three interfaces**.
- Therefore, C1 must provide implementations for all three methods:
 - **display()**
 - **play()**
 - **show()**
- **obj.display()** prints the value of **x**, which is **20**.
- **obj.play()** prints the message from interface **I2**.
- **obj.show()** prints the value of **z**, which is **30**.
- **obj.y** accesses the variable **y** from interface **I2**, which is **50**.

Structure



Output

```
20  
It is from interface 2  
30  
50
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> javac C1ImpI1I2I3.java  
PS C:\Users\navee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> java C1ImpI1I2I3  
20  
it is from interface 2  
30  
50
```

4-C1 EXTENDS C2 IMPLEMENTS I1 I2 I3

```
interface I1 {
    void method1();
}

interface I2 {
    void method2();
}

interface I3 {
    void method3();
}

class C2 {
    void show() {
        System.out.println("This is C2");
    }
}

class C1 extends C2 implements I1, I2, I3 {

    public void method1() {
        System.out.println("I1 method");
    }

    public void method2() {
        System.out.println("I2 method");
    }

    public void method3() {
        System.out.println("I3 method");
    }
}

public class C1ExtC2ImplI1I2I3 {
    public static void main(String[] args) {

        C1 obj = new C1();
    }
}
```

```

        obj.show();
        obj.method1();
        obj.method2();
        obj.method3();
    }
}

```

Explanation

There are **four types** involved:

- **C2** → Parent class
- **C1** → Child class
- **I1, I2, I3** → Interfaces

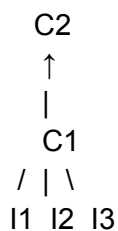
The important line is:

class C1 extends C2 implements I1, I2, I3

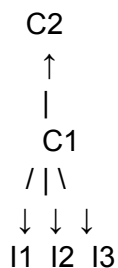
It means:

- **extends C2** → C1 inherits from class C2.
- **implements I1, I2, I3** → C1 implements all three interfaces.
- C1 gets the **show()** method from C2.
- C1 must implement **method1()**, **method2()**, and **method3()** because they are declared in the interfaces.

Structure



More accurately:



Main Method

```
C1 obj = new C1();
```

Creates an object of C1.

```
obj.show();
```

Calls `show()` inherited from **C2**.

```
obj.method1();
```

Calls the implementation of **I1**.

```
obj.method2();
```

Calls the implementation of **I2**.

```
obj.method3();
```

Calls the implementation of **I3**.

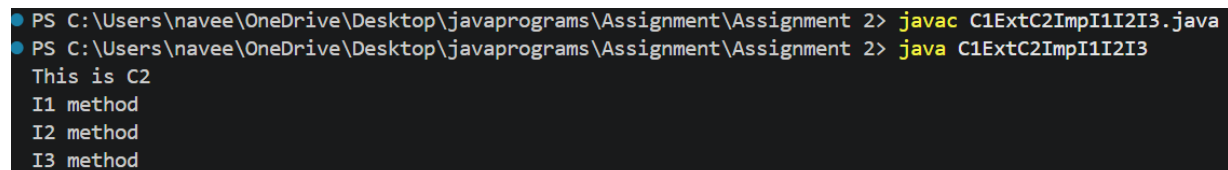
Output

This is C2

I1 method

I2 method

I3 method



```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> javac C1ExtC2ImpI1I2I3.java
PS C:\Users\navee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> java C1ExtC2ImpI1I2I3
This is C2
I1 method
I2 method
I3 method
```

5-I1 EXTENDS I2

```
interface I1 {
    int x = 20;

    void show();
    void display();
}
```

```
interface I2 extends I1 {
```



```

    int y = 50;

    void show();
    void play();
}

class A implements I2 {

    public void show() {
        System.out.println("It is implemented from interface");
    }

    public void play() {
        System.out.println("y value from I2 " + y);
    }

    public void display() {
        System.out.println("Implemented from I1");
    }
}

public class Interfacetointerface {

    public static void main(String[] args) {

        A obj = new A();

        obj.display();
        obj.show();
        obj.play();
        System.out.println(obj.x);
    }
}

```

Explanation

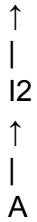
The important line is:

interface I2 extends I1

This means **I2 inherits from I1**.

Structure:

I1



- **I1** contains:
 - `x = 20`
 - `show()`
 - `display()`
- **I2 extends I1**, so I2 inherits the members of I1.
- I2 additionally contains:
 - `y = 50`
 - `play()`
- **class A implements I2** means A implements I2 and also gets the methods inherited by I2 from I1.
- Therefore, A implements:
 - `show()` → from I1/I2
 - `display()` → from I1
 - `play()` → from I2

Why is `show()` written in both I1 and I2?

```
interface I1 {  
    void show();  
}
```

```
interface I2 extends I1 {  
    void show();  
}
```

This is allowed because the method has the **same signature**. I2 is redeclaring the inherited method. Class A only needs to provide **one implementation** of `show()`.

Main Method

```
A obj = new A();
```

Creates an object of class A.

```
obj.display();
```

Calls the implementation of `display()` from I1.

```
obj.show();
```

Calls the implementation of `show()`.

```
obj.play();
```

Calls the implementation of `play()` from I2.

```
System.out.println(obj.x);
```

Prints the value of `x` inherited from I1.

Output

Implemented from I1

It is implemented from interface

y value from I2 50

20

```
PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> javac InterfacetoInterface.java
PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> java InterfacetoInterface
implemts from i1
it is implements from interface
y value from I250
20
```

6-I1 EXTENDS I2 I3

```
interface I2 {
    void showI2();
}
```

```
interface I3 {
    void showI3();
}
```

```
interface I1 extends I2, I3 {
    void showI1();
}
```

```
class C1 implements I1 {

    public void showI1() {
```

```

        System.out.println("This is I1");
    }

    public void showI2() {
        System.out.println("This is I2");
    }

    public void showI3() {
        System.out.println("This is I3");
    }
}

public class I1ExtI2I3 {
    public static void main(String[] args) {

        C1 obj = new C1();

        obj.showI1();
        obj.showI2();
        obj.showI3();
    }
}

```

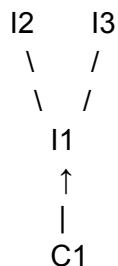
Explanation

The important line is:

interface I1 extends I2, I3

This means **I1 inherits from both I2 and I3**.

Structure:



- I2 contains `showI2()`.
- I3 contains `showI3()`.
- I1 extends I2, I3, so I1 inherits both `showI2()` and `showI3()`.
- I1 also has its own method `showI1()`.

- C1 implements I1, so C1 must implement all three methods:
 - showI1()
 - showI2()
 - showI3()

Main Method

C1 obj = new C1();

Creates an object of C1.

obj.showI1();

Calls the method from I1.

obj.showI2();

Calls the method inherited by I1 from I2.

obj.showI3();

Calls the method inherited by I1 from I3.

Output

This is I1

This is I2

This is I3

```
PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> javac I1ExtI2I3.java
PS C:\Users\navvee\OneDrive\Desktop\javaprograms\Assignment\Assignment 2> java I1ExtI2I3
This is I1
This is I2
This is I3
```